

Basic Computing at qBio11*

Peter Carbonetto *University of Chicago*

The aim of this tutorial is to introduce R and to use R to analyze a biological data set. We will analyze the data interactively while also developing Good Practices for data analysis, including writing an *R script*. This workshop is intended for biologists with little to no background in programming. [This tutorial is also a Google doc.](#)

How this tutorial is organized

Basic Computing is split into 3 parts:

1. Understanding R, the real fundamentals
2. Understanding R, working with more complex data structures
3. An exploratory data analysis

What is this document, and how should I use it?

This is an R Markdown document and a Google doc (bit.ly/3Ulll7s) containing text and R code. You can run the code by copying & pasting it into your favorite R IDE (“integrated development environment”). Here we will use the RStudio IDE. (Popular online IDEs include Posit Cloud and Google Colab.)

In RStudio, we will mainly use two “panes”: the “Source pane” and the “Console pane”.

For some practice, try running these two lines of code:

```
x <- rnorm(200)
hist(x, n = 32)
```

You may notice that your histogram is not the same as mine. *Why is that?*

Make this document your own by making a copy of it in your Google Drive, then add your own notes and code. And of course use the workbook for your notes. I’ve also given you permission to add comments to this Google doc.

Part 1: Understanding R, the real fundamentals

Now I want us to revisit R from the ground up. Some of you may be already familiar with R or perhaps other programming languages. Even if you are already familiar with R, sometimes it is

*This document is included as part of the Basic Computing—Introduction to R tutorial packet for the BSD qBio Bootcamp, University of Chicago, 2025. **Current version:** August 11, 2025; **Corresponding author:** pcarbo@uchicago.edu. Thanks to Stefano Allesina, John Novembre, Stephanie Palmer and Matthew Stephens for their support and guidance.

helpful to revisit and strengthen your understanding of the programming language. Later on, we will analyze a real data set and apply our R skills to that data set. But for now I want to focus on building skills and getting comfortable with the R environment.

R is like math

There is a close relationship between math and programming languages: many or most expressions in R look very much like mathematical expressions. Let's take this expression as an example:

```
sqrt((x1 - x2)^2 + (y1 - y2)^2)
```

What does this expression do? Break down the mathematical operations step-by-step.

But, when you try to evaluate this expression, you will get an error that "x is not found".

Let's define x and y and try to evaluate that expression again:

```
x1 <- 1
y1 <- 1
x2 <- 2
y2 <- 2
sqrt((x1 - x2)^2 + (y1 - y2)^2)
```

Let's just use our intuition here to write with another expression in R, the distance between 2-d points x and y in "taxicab geometry":

```
abs(x1 - x2) + abs(y1 - y2)
```

R's single most powerful feature: making your own functions is easy

Functions in R are everywhere. We have used at least two already, `sqrt` and `abs`. (Side note: almost every function in R is documented. If you want to know what a function does and how to use it, simply type `help(name_of_function)`, e.g., `help(sqrt)`.) But you aren't limited by R's existing functions: R is easily extensible by *creating your own functions*. For example, it is tedious to write the distance between two 2-d points every time you want to do this. One way to make this calculation more efficient is to create a new function that performs this calculation inside it and then returns the result:

```
euclidean_dist <- function (x1, y1, x2, y2) {
  d <- sqrt((x1 - x2)^2 + (y1 - y2)^2)
  return(d)
}
```

We have created a new object in our environment *that is a function* (yes, functions in R are objects, a fact that is also very powerful, but something we will not explore immediately):

```
ls()
euclidean_dist
```

Now that we have created this function, we can use it on its own, or by combining it with other mathematical expressions:

```
euclidean_dist(1,1,2,2)
euclidean_dist(-1,-1,1,1)
euclidean_dist(6*7,4,3/4,exp(1))
sin(euclidean_dist(1,1,2,2))
```

You will create functions in the other tutorials and workshops.

Exercise: Create a function called “taxicab_dist” to compute the taxicab distance between two points in 2 dimensions, and then perform some calculations with it.

Discuss: Why was it useful or why could it be useful to create a function to perform your calculations?

Followup discussion: Custom functions are one of the most underused and underappreciated features of R. Why do you think this feature is so underused?

Two evaluation modes in R

There are two ways that R evaluates expressions: *calculator mode* and *environment building mode*.

We have actually seen examples of both. Let’s revisit the lines of code we have evaluated to understand the differences between the two modes.

Data vectors

Perhaps without knowing it, we have already been working with data vectors: for example, the object `x1` is a vector of length 1 (sometimes called a “scalar”).

Almost everything in R works on *vectors* of any length. So once you get comfortable making calculations on single numbers (“scalars”), it is often very natural to apply your same calculations to vectors. (In fact, over time, you will find it so natural that you may take it for granted!) In order to work with data vectors, it is helpful to know a few useful functions for *creating* vectors.

The most flexible function is the “combine” function, or “c” for short:

```
x <- c(0, 7, 1e-6, -50, 2^5, 1/3, 2*pi)
```

Another useful function is “rep”, which can be used to quickly create very large data vectors:

```
x <- rep(1, 100)
```

And “seq”, short for “sequence”:

```
x <- seq(1,100)
x <- seq(1,100,5)
```

These functions can (of course!) be used in combination in many different ways, e.g.,

```
x <- rep(seq(1,3),10)
x <- c(seq(1,10),-1,rep(0,4))
```

Another interesting function for creating data vectors creates random data vectors:

```
x <- sample(10)
x <- sample(1000,10)
x <- sample(seq(0,100,2),10)
```

Use the square brackets to access an individual element of a data vector:

```
x[1]
x[2]
```

And “length” to get the number of elements in the vector:

```
length(x)
```

There is also a special vector in R, the “null vector”, which is a vector of length zero:

```
x <- NULL
length(x)
```

Surprisingly, the “null vector” can be useful! (You will see an example of this later.)

R operates on data vectors in two different ways: (i) by applying the calculation separately to each element of the vector, or (ii) by aggregating across the elements. Here are some examples of the first way:

```
x <- seq(1,10)
x / 10
sqrt(x)
abs(x)
x + 1
y <- sample(10)
x + y
abs(x - y)
```

Here are some examples of the second way:

```
sum(x)
mean(x)
max(x)
min(x)
which.max(x)
which.min(x)
```

These last two examples are rather curious... *what is the value returned from the last two examples?*
And here are some examples that combine the two ways:

```
mean(x - y)
max(abs(x - y))
```

This is R's **invisible magic**. *Why do I call it R's "invisible magic"?*

In-class activity: random walks

Through an in-class activity, we will explore these distance calculations further, and in the process learn about **loops**, **if-then-else**, and how to simulate data in R and store the simulated data in vectors.

We will work on this activity by writing an **R script**. The first step is to create a new .R file in RStudio and give it a useful name that you will remember (e.g., random_walk.R).

A powerful and ubiquitous idea in quantitative biology is the design of simulations to understand natural phenomena. One of the simplest and oldest simulations is the "random walk", also sometimes called the "drunkard's walk". (Incidentally, I highly recommend the book by Leonard Mlodinow, [The Drunkard's Walk: How Randomness Rules Our Lives](#).) Here we will simulate a 1-d random walk.

Here is a script to get you started:

```
n <- 100
x0 <- 0
x <- x0
locations <- NULL
times <- seq(1,n)
for (i in times) {
  coin <- sample(2,1)
  if (coin == 2) {
    x <- x + 1
  } else {
    x <- x - 1
  }
  locations <- c(locations,x)
}
plot(times,locations,type = "l")
```

First exercise: Add some code to your script to determine the time at which the random walk is furthest away from the starting point. (Which distance will you use? Euclidean distance, taxicab distance, ...?)

Second exercise: Use R's "invisible magic" to simplify the calculation of the largest distance to the starting point, and the time in which it occurred. I recommend creating a new R script for this exercise so that you can compare your previous distance calculation to the new one. (To verify that you are getting the same result, you can add a `set.seed(1)` to your script to make the results "reproducible".)

Third exercise (time permitting): Write another script, e.g., `random_walk_2d.R`, to simulate a 2-d random walk using a 4-sided die. Similar to before, using R's "invisible magic", determine the the largest distance to the starting point, and the time in which it occurred.

Part 2: Understanding R, working with more complex data structures

In this part we will learn about some more complex data structures and how to work with them through the analysis of a real data set from a paper published in *Genetics*.

My data analysis

Here is my data analysis. It is an analysis of data from a [2008 Genetics article](#) on the genetics of dog breeds:

```
dogs <- read.csv("dogs.csv", stringsAsFactors = FALSE)
fit <- lm(aod ~ weight, dogs)
print(fit)
```

You will notice that the code is very short! There's nothing wrong with that—successful analyses in R do not need to be long or complicated! Still, we will spend quite some time understanding this code and what it does, and in the process we will learn about R. But before we try to understand what the code is doing, let's start by trying to run the code in RStudio to reproduce the result.

A mini-course on data frames

A data frame is R's main data structure for *storing tabular data*. The data frame is one of the most important data structures in R, and is important enough that we will spend much of this tutorial seeking to understand how to work with and analyze data in data frames. (Not all data of course is tabular data, but because so many things in R work well with data frames, *it can be helpful to find ways to rework your data so that it fits into a data frame.*)

Before we get to more interesting things, we need to first get comfortable with some basic syntax for data frames.

```
print(dogs)
head(dogs)
tail(dogs)
```

```

summary(dogs)
class(dogs)
nrow(dogs)
ncol(dogs)
names(dogs)
dogs$breed
dogs$aod
dogs$height
dogs$weight
dogs[, "aod"]
dogs[1,]
dogs[, "weight"]
dogs[1, "weight"]
x <- dogs[1, "weight"]
x <- dogs[, "weight"]
x <- dogs[1,]

```

You may have noticed that many of these lines of code *do the exact same thing*. This is a common theme in R (and in programming more generally): *there are often many different ways of accomplishing the same thing, and there is rarely one way that is “best”* (although sometimes R programmers get into passionate debates about this).

A reminder at this point that the advantages of analyzing data in R may be less obvious when working with a small a data set. Later when we work with a large data set the benefits of R will become more clear.

Checking for mistakes

All researchers, no matter how much experience they have, make mistakes. And most mistakes are stupid mistakes! (I have made my fair share of embarrassing mistakes.) Occasionally, [very good researchers make mistakes in their papers](#). (See also [here](#) for a discussion of this mistake years later.) Therefore, *try to catch your mistakes early*.

So how can you find your mistakes? The good news is that R catches many of your mistakes: most methods in R have their own internal error-checking. Discussing your results with your labmates and advisors is also another way to catch mistakes. But ultimately you will need to develop some skills and strategies for performing your own checks.

A relatively simple but nonetheless helpful check is a “sanity check”: it doesn’t tell you the result is right, but at least it reassures you that the result is not horribly wrong. A good sanity check is simple—one that you can do on “the back of an envelope”. Here we will perform a simple sanity check for our analysis of the dogs data (and in the process we will learn more about R).

Here our “sanity check” will be to hand-check that our model, $y = ax + b$, where x = weight and y = aod, is making sensible predictions of y given x . This will involve some basic arithmetic so in the process we will learn about how to do arithmetic in R. Also, plots are another powerful way to perform checks, and we will write some simple code to visualize our results.

 Your code for checking your results.

In-class activity: Re-run this analysis on a different data set

Let's now try something *a bit audacious*. Let's try to re-run the same analysis as before, but on [a different data set from the American Kennel Club](#). Will this new analysis produce a similar result, or not?

Using the skills you have developed so far—and a bit of creativity—I believe you can adapt the code you ran on the “dogs” CSV file to analyze the AKC data (the file is `akc_data.csv`). This exercise will involve making some judgments about how to analyze the data and therefore I do not expect everyone to get the same result. It will also involve some initial explorations of the data to understand what this data set contains, and how it differs from the first data set. *Do not be afraid to make mistakes!*

Note that you may encounter a challenge that you have not yet dealt with. When you encounter this challenge, try to figure out what is the issue, and we will discuss as a group how to overcome it.

 Your code for analyzing the AKC data.

Tending to your gaRden

(Almost) every line of code in R acts on your R environment: it takes objects that exist in your environment, then generate new objects or overwrites existing objects. Therefore, to understand any line of code, you need to understand not only the code itself, but also what is the state of your environment (your “garden”) the moment before you run your code; that is, the objects that are in your environment and what they represent. However, this is easier said than done when your environment (garden) is messy (not tended to). Therefore an important skill as a coder is to tend to your gaRden.

 Your code for tending to your gaRden.

What are all these objects? Could we have named some of these objects better to remind us what they are?



S. cerevisiae

Programming challenge: facts about dog breeds

The data frame is R's way of storing tabular data. It is one of the most important and more powerful data structures in R. In this Programming Challenge, we will take some time to understand data frames and how to work with them. Although our focus here is on data frames, many of the ideas you will pick up in this Programming Challenge are quite general.

Let's start this part of the tutorial with a clean environment. Then go ahead and import the dogs data set and print out the first few rows of the table:

```
rm(list = ls())
dogs <- read.csv("dogs.csv", stringsAsFactors = FALSE)
head(dogs)
```

The data frame is a data structure for storing tabular data. The data are actually stored in a very specific way: the data frame is a set of columns, and each column is a vector of the same length. Let's run some code to convince ourselves of this fact.

First, for convenience, make a copy of the first column, and call it "x":

```
x <- dogs$breed
```

Important note: This makes a copy of the original data, so if you were to modify or delete x, this leaves "dogs" unchanged.

This is a "character" data type. It is R's way of storing text data:

```
class(x)
length(x)
x
```

Note we could have also copied the first column this way:

```
x <- dogs[, "breed"]
```

And here's another way!

```
x <- dogs[, 1]
```

Which way do you prefer?

By storing tabular data in this way, we can *divide and conquer*: since each column is also an object in its own right, if the data are too complicated to understand all at once, we can make a copy of

the columns we want to look at more closely, and run code on the copy. *This is a useful strategy for dealing with complex data sets.*

To drive home this idea of a data frame as a collection of vectors, the way to create a data frame is in fact to join together a bunch of vectors of the same length. For example:

```
mydogs <- data.frame(  
  breed = dogs$breed,  
  lbs = dogs$weight,  
  years = dogs$aod)  
head(mydogs)
```

Each data structure in R has its own features and its own techniques for working with them. In time, you will learn to work with other types of data structures: some are used widely (e.g., an “lm” object), and some are very specialized (e.g., a [GRanges object](#)).

Now that we have some basic understanding of what is a data frame, let’s jump into the Programming Challenge: the goal is to write code to answer some basic questions about dog breeds from the data. The challenges will get progressively more difficult, and will build on each other, so try not to rush through them. Sometimes the code will be given to you, other times you will be given hints for writing the code to answer the questions. Now, this data set is small enough that you can answer many of these questions by eye, but please don’t do that. (That being said, you are welcome to look at the data to verify your answers.)

Before diving deeply into the Programming Challenge questions, first discuss a collaboration strategy with your teammates. How will you work on the problems together? How will you share and discuss solutions? (Maybe do this in a shared Google doc?) How will you make sure that everyone is included in the problem solving? How will you address conflicts, e.g., when your team comes up with more than one solution?

Warmup: Smallest and largest dog breeds

This should find the average height (in inches) of the largest dog breed:

```
x <- dogs$height  
max(x)
```

Now write code to find the (average) height of the smallest dog breed:

 Add your code here.

This didn’t tell us which breeds were the smallest and largest. For example, to find the largest breed, we can do this:

```
y <- dogs$breed
i <- which.max(x)
y[i]
```

The output of `which.max()` was stored in object “i”. How kind of object is “i”?

What is the smallest breed?

 Add your code here.

What objects did you create to answer this question?

Another warmup: inspecting data about specific dog breeds

Suppose you wanted to look more closely at the height, weight and other statistics of the Alaskan Malamute, which is stored in the fifth row of the data frame. This is easily done by selecting the fifth row with the square brackets:

```
dogs[5,]
```

You can also select several rows at once, e.g.,

```
dogs[c(43,46),]
```

Practice a few times selecting different combinations of rows. What happens if you select a row number that is larger than the height of the table?

Facts about dogs’ BMI

The body-mass index (BMI) is a standard quantity—and sometimes misused quantity!—in science and medicine. In R, the BMI is easily calculated:

```
w <- dogs$weight
h <- dogs$height
bmi <- 703*w/h^2
```

Based on this code, what is the mathematical formula for BMI?

Next, write some code to find the largest, smallest, mean and median BMI. What are the dog breeds with the largest and smallest BMI?

 Add your code here.

If you would like to use the BMI data later on, you can insert these data in the data frame:

```
dogs$bmi <- bmi
```

(What is the benefit of adding these data to the data frame as opposed to storing the BMI data in a separate object?)

The longest-living dog breeds

In the dogs data frame we encountered two types of data: numeric data and text data. Another type of data that is very important is *logical data*. Although the data frame does not contain logical data, we can easily create logical data using *logical operators*.

You might create logical data in the process of answering questions about the data. For example, suppose you would like to know how many dogs have an expected longevity of 16 years or greater. Here is some code to answer this question:

```
x <- dogs$aod >= 16
summary(x)
```

What is “x” here?

```
class(x)
x
```

To get the indices that are “TRUE”, use which():

```
i <- which(x)
length(i)
i
dogs[i,]
```

What does “i” contain?

Other logical operators include equals (==), and (&), or (|) and not (!). Running help(Logic) will give you a longer list.

Now write similar code to find the breeds with the following characteristics:

1. Expected age of death (AOD) at least 15 and average weight greater than 20 lbs.
2. AOD greater than 15 and “shortcoat” value of 1. (Later we will learn what the “shortcoat” column represents.)

 Add your code here.

A QTL for weight (also, dealing with missing data)

In the *Genetics* paper, the strongest QTL (“quantitative trait locus”) for weight was a QTL on chromosome 7. The “cfa7_46696633bp” column stores the breeds’ allele frequencies for these QTL. What happens when you try to calculate the correlation between the allele frequencies and weights?

```
x <- dogs$cfa7_46696633bp
y <- dogs$weight
cor(x,y)
```

It turns out that the allele frequencies were not available for some of the breeds, so a special value “NA” was entered for those breeds. *Why “NA”?*

Use the `is.na()` function, and other functions you have used before, to find the missing entries, and determine:

1. How many allele frequencies are missing?
2. Which breeds are missing allele frequencies?

 Add your code here.

The designers of R, appreciating that missing data is widespread in statistics, made sure that missing values were an integral part of the R programming language. Most statistical functions in R can deal with missing data. Read the documentation for “mean” and “cor”, then use the guidance provided in the documentation to compute the average allele frequency at the QTL (that is, averaged across all dog breeds), and the correlation between body weight and allele frequency.

 Add your code here.

What is a “factor”?

So far, we have seen three basic data types: character, numeric and logical. There is a fourth important atomic data type in R: *factor*. What is unusual about factors is that—last I checked—

there is no equivalent in other popular programming languages, at least not as a primitive data type. And yet you will find that they are extremely useful.

None of the columns in the dogs data frame are a factor. But, like logical data, *we can create a factor from other data*.

The “shortcoat” column contains numeric data. As it turns out, *it may be more useful to analyze the data as a factor*.

In this part of the Programming Challenge, you will be given the code, and your task will be to run the code and interpret the outputs, with the aim of gaining some intuition for factors, how to use them, and when they may be useful.

Let’s first run a few lines of code to inspect the shortcoat data:

```
x <- dogs$shortcoat
class(x)
x
summary(x)
unique(x)
```

Now run the following lines of code to create a factor:

```
x <- factor(x)
class(x)
x
summary(x)
```

Notice that although the data have stayed the same, the two summaries are different. This is because although the data haven’t changed, the *data representation or encoding* has: *R treats the numeric encoding differently from the factor encoding*.

What does the second summary tell us? Judging by these outputs, what do you think a factor is?

These observations suggest that we can improve the data representation further; the data should be as easy to interpret as possible (“human readable”). Representing the data as zeros and ones may be convenient for the computer, but it is confusing when representing the data as a factor because it “looks” like numeric data, when in fact it is not.

Fortunately, now that the data are stored as a factor, this is easily fixed. To fix this, we modify a property (“attribute”) of the object. Since this is the first time we are using an object’s attributes, let’s first take stock of the object’s attributes:

```
attributes(x)
```

What does the “levels” attribute keep track of?

We can modify the levels attribute. For example, this replaces the zeros with “no” and ones with “yes”:

```
levels(x) <- c("no", "yes")
summary(x)
```

Having made these improvements to the shortcoat data, let's store the improved data in the data frame:

```
dogs$shortcoat <- x
summary(dogs)
```

To illustrate the power of factors, let's see how easily it can be incorporated into a linear regression analysis.

In one of my analyses, I found that dogs with short coats tended not to live quite as long as dogs with longer coats:

```
sx <- dogs$shortcoat
y <- dogs$aod
i <- which(x == "no")
j <- which(x == "yes")
mean(y[i])
mean(y[j])
```

Is this difference significant? We can check this with `lm()`:

```
fit <- lm(aod ~ shortcoat, dogs)
coef(fit)
summary(fit)
```

What does the “shortcoatyes” output from `coef(fit)` represent? Is the difference significant?

The difference in expected lifespan might be explained better by differences in the body weights between dogs with short and long coats. Is the AOD difference explained by “shortcoat” still significant when weight is included as an explanatory variable for AOD?

```
fit <- lm(aod ~ weight + shortcoat, dogs)
summary(fit)
```

An index of dog breeds

There are many situations in biology research in which your data are text data (consider that DNA sequences are a type of text data). The `stringi` and `stringr` packages are popular packages for performing more complex analyses of text data. (In computer science, a “string” means a sequence of characters, so one can think of text data as a collection of strings.) In this last question, we will practice some simple manipulation of text data (“strings”) to organize the dog breeds by the first letter of the breed name.

The first step is to extract the data we want. This can be done using `substr()`:

```
x <- dogs$breed
d <- substr(x, start = 1, stop = 1)
d
```

Which is the most common first letter for a dog breed? And how many breeds start with this letter? To answer these questions, try creating a factor.

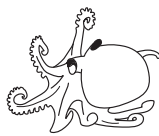
 Add your code here.

The `factor()` function automatically determined which letters appeared in the data. But it could be useful to include all the letters in the alphabet, not just the ones that appear in the data. Modify your factor call above to include the unused letters as well. See `help(factor)` and `help(LETTERS)` for guidance.

 Add your code here.

Once you have the new factor using all the letters, write code to determine which letters are not used for the first letter of any dog breed:

 Add your code here.



O. bimaculoides

Part 3: An exploratory data analysis

Here's the situation. We have generated some data from mouse pancreas samples. We would like to use these data to generate some exciting scientific breakthroughs (obviously!). But before we do that, we need to understand the data. This includes checking our data for errors, and exploring the data. *We will perform all of these steps in R.*

Setup instructions

1. Create a “project folder” (e.g., `pancreas_explorations`) where we will store all of the files related to our data analysis, including the data, code and results.
2. In RStudio, change your working directory to the folder you just created. *What is the “working directory” and why do we need to change it?*
3. Create a new R script and give it a memorable name, e.g., `pancreas_analysis.R`.
4. Add a brief description in comments at the top of the R script, e.g., “This is the code implementing my initial analysis of the pancreas data.”
5. Download the data files from Box (bit.ly/4m6pgSS). You should have three files.

The CSV files

Two of the files you downloaded are CSV files. *What is a CSV file, and what is “CSV” short for? What kind of data can be stored in a CSV file? Why does the CSV file remain a popular way to store data despite the fact that there are more sophisticated alternates (e.g., Excel, HDF5)?*

The R script

Here we will write code to analyze the data and generate results. The code is as important—or perhaps more important!—than the results themselves.

Discussion: Why is the code important? What role(s) does it play in your research?

Important note: In this part, we will spend less time understanding the R code. Instead, I want to focus more on the practice the coding and developing the data analysis, which involves a cycle of (i) investigating the data, (ii) leading to a better understanding the data, and then (iii) motivates us to revise our analysis.

Import the data into R

Note: put this code in the R script first.

```
barcodes <- read.csv("pancreas_barcodes.csv")
genes <- read.csv("pancreas_genes.csv")
counts <- readRDS("pancreas_counts.rds")
```

Internalizing the data and the variables in our computing environment

Here we have three variables so far (soon we will create more). Our immediate goal is to run some code that will help us understand—and *internalize*—these variables.

```
ls()
print(barcodes)
class(barcodes)
```

```
nrow(barcodes)
ncol(barcodes)
head(barcodes)
tail(barcodes)
summary(barcodes)
barcodes$replicate
table(barcodes$replicate)
```

Now do the same for the “genes” data frame.

The “counts” variable is a different type of data structure—it is a matrix—so we will want to examine it a little differently.

```
class(counts)
nrow(counts)
ncol(counts)
counts[1:3,1:3]
counts[,1]
counts[1,]
min(counts)
max(counts)
```

Now one thing to note here is that these three variables represent three sets of data, but they are also closely connected.

Indeed, this sort of setup is so common in biology that it has a name and there is software developed specifically to analyze these sorts of data: “annotated data matrix”, or “anndata” for short.

The row and column numbers match, but do the actual rows/columns match? First check the genes:

```
head(genes)
counts[1:3,1:6]
```

Anecdotally, it appears so.

The barcodes are harder to check, though:

```
head(barcodes)
counts[1:6,1:3]
```

How can we *automate* this check? This involves slightly more complicated R code:

```
barcodes$barcode == rownames(counts)
all(barcodes$barcode == rownames(counts))
```

Now take some of this code that we have used to explore the data and add it to the script to print out basic stats on the data set (here are provide some sample code, but it doesn’t have to be exactly this).

```
cat("number of cells:\n")
print(nrow(counts))
cat("number of genes:\n")
print(ncol(counts))
cat("number of cells from each replicate:\n")
print(table(barcodes$replicate))
```

Thinking more about the data and our data exploration aims

What are these data, and what should we do with them? A few thoughts:

1. Each row in the matrix is a gene expression profile of an individual cell in the pancreas at a moment in time.
2. Let's stop and think how incredibly powerful this is: being able to capture the state of a single cell, and gather a massive amount of information of a single cell, was something unimaginable a few years ago. The technology is called "single-cell RNA-sequencing", but really it is not one technology, but rather the accumulation of many scientific and engineering breakthroughs
3. The individual entries of the matrix give the number of "reads" for that gene. It is a measurement of the level of gene transcription in that cell at that point in time.
4. What is the purpose of collecting these data? The exciting possibility is that we can use these data to reconstruct the pancreas from the ground up. This reconstruction should, first of all, confirm our existing understanding of the pancreas. But perhaps it will show us something unexpected and new discoveries—perhaps it will reveal are some types of cells in the pancreas that we did not know existed? Or perhaps some cells that are acting in unexpected ways? How to make sense of these data remains an unresolved question in the scientific community. But we can still try to take some simple steps to extract some understanding from the data, and perhaps even discover something interesting.
5. Single-cell RNA-sequencing produces a huge amount of information. But more is not always better; we need to also be on guard for poor quality, misleading, irrelevant, or "noisy" data.
6. It is much easier to assess the data when have some idea of what to look for. So let's start with some things that we know about the pancreas before we try to discover completely new things. (Also, it is helpful to check that the data confirms our basic expectations about the pancreas—if it doesn't, that might be a cause for concern!) For example, we should find acinar and ductal cells that produce and move the pancreas's digestive enzymes. We should also find different types of islet cells such as α and β cells which are the endocrine ("hormone-producing") cells in the pancreas (the β cells in particular produce insulin). Do these data contain gene measurements from these sorts of cells?

Run this code first in the console, then add some of it to the script.

```
colnames(counts) <- genes$symbol
counts[, "Krt19"]
x <- counts[, "Krt19"]
hist(x, n = 64)
```

Second attempt:

```
x <- log10(counts[, "Krt19"])
hist(x, n = 64)
```

Third attempt:

```
x <- log10(counts[, "Krt19"] + 1)
hist(x, n = 64)
```

Why take the log-counts? What are the implications? Why use the base-10 logarithm (as opposed to, say, base-2 or base-e)?

Let's now search for the insulin-producing β cells:

```
x <- log10(counts[, "Ins1"] + 1)
hist(x, n = 64)
```

Add some of this code to the R script to keep track our progress in exploring these data.

Data-driven discovery

So far, we have searched for cells based on prior knowledge of the pancreas. Can we instead make discoveries *using the data alone*? *How do we do that?*

Typically, we use statistical or machine learning techniques that identify patterns or structure from data. Two very commonly used techniques are PCA (principal component analysis) and clustering. Another more recent technique is a *nonlinear embedding* technique called UMAP. The intuition behind UMAP is that if some cells have similar gene expression profiles, the UMAP will “project” these cells nearby each other on a 2-d map, and so they will appear as a “blob” or “cluster” in 2-d space. But of course biology is complicated... so let's see what happens.

There is also a *computational challenge* in working with these data: it is *a lot* of data, so we will need to use advanced numerical techniques to work with data of this size.

Reducing the size of the data using PCA

The first “trick” is to use a very simple technique, PCA (equivalently, SVD), to reduce the size of the data so that it is more manageable. But even running PCA is challenging on a data set of this size, so we will use a randomized algorithm implemented in an R package called “rsvd”.

```
library(rsvd)
set.seed(1)
shifted_log_counts <- log(counts + 1)
pca <- rpca(shifted_log_counts, k = 30)
```

Out of curiosity, let's plot the top two PCs:

```
x <- pca$x[,1]
y <- pca$x[,2]
plot(x,y,pch = 20)
```

Note that the output is a more complex data structure called a “list”:

```
class(pca)
names(pca)
class(pca$x)
nrow(pca$x)
ncol(pca$x)
pca$x[1:4,1:4]
```

Computing a nonlinear 2-d embedding of the cells

Here we will use the “uwot” R package to run UMAP:

```
library(uwot)
set.seed(1)
res <- umap(pca$x,metric = "cosine",n_neighbors = 30,
            min_dist = 0.3,verbose = TRUE)
```

The “umap” function is an example of a complex function with many input arguments, and learning to use this function well is a “mini-expertise” in itself. Learning to use this function well is obviously beyond the scope of this tutorial. (I’ll just note that in R it is easy to access the documentation for this function with the “help” feature.) Although seemingly overwhelming in its complexity, most of the inputs to the umap function have sensible defaults that are expected to work well a lot of the time. But a more in-depth understanding might be needed if this result is important for your work and you are not satisfied with the initial result. (As you can see, in our code we did override some of the defaults.)

The result from umap is a matrix, which we are already a little bit familiar with:

```
class(res)
nrow(res)
ncol(res)
head(res)
```

Let’s create a plot similar to the PCA plot above:

```
x <- res[,1]
y <- res[,2]
plot(x,y,pch = 20,xlab = "UMAP 1",ylab = "UMAP 2")
```

The UMAP plot appears to be revealing tons of interesting structure, including small and large “blobs” of cells, some that are quite disconnected from the others.

Note: Later we will learn better ways to create plots using the ggplot2 package.

Save the results

Let's call this version 1.0 of our data analysis.

Your advisor has asked to bring the results of your analysis to your one-on-one with him next week. What should you bring? You have already diligently saved your code. What about the results—what results should you save? Write some code to save the UMAP results:

```
plot(x,y,pch = 20,xlab = "UMAP 1",ylab = "UMAP 2")
dev.copy2pdf(file = "pancreas_umap.pdf",height = 4.5,width = 4)
write.csv(res,"pancreas_umap.csv",quote = FALSE)
```

Your R script might look something like this (this is the minimal version):

```
library(rsvd)
library(uwot)
set.seed(1)
barcodes <- read.csv("pancreas_barcodes.csv")
genes <- read.csv("pancreas_genes.csv")
counts <- readRDS("pancreas_counts.rds")
s <- rowSums(counts)
i <- which(s > 5000)
barcodes <- barcodes[i,]
counts <- counts[i,]
s <- rowSums(counts)
s <- s/mean(s)
shifted_log_counts <- log(counts/s + 1)
pca <- rpca(shifted_log_counts,k = 30)
res <- umap(pca$x,metric = "cosine",n_neighbors = 30,
            min_dist = 0.3,verbose = TRUE)
x <- res[,1]
y <- res[,2]
plot(x,y,pch = 20,xlab = "UMAP 1",ylab = "UMAP 2")
```

Exercise: Test your script by running it in a “clean” environment.

A wrinkle in the analysis

Data analyses are always iterative: an analysis is gradually improved over time with careful review, feedback, literature review, etc. From discussions with your advisor in the one-on-one meeting, you learned that you made a big mistake on a step that we didn't give much thought to (it is the part where we created `shifted_log_counts`): what matters is not the raw gene expression levels, but instead the *relative levels*. In other words, we need to be comparing *proportions* rather than counts. This reconsideration also brings to light another hidden problem lurking in these data: some of the cells have much less expression the others suggesting some technical issues in getting the data for these cells, and therefore maybe it would be better not to include them in our analysis.

```
s <- rowSums(counts)
hist(log10(s),n = 64)
```

Maybe we will get more trustworthy results if we remove these cells with low total expression.

Fortunately, because we have carefully developed an R script, we do not need to start from scratch—we only need to make a few changes to our script and re-run the script.

```
i <- which(s > 5000)
counts <- counts[i,] # Dangerous!
barcodes <- barcodes[i,]
```

Check the total expression levels after filtering:

```
s <- rowSums(counts)
hist(log10(s),n = 64)
```

Now compute the expression proportions:

```
s <- rowSums(counts)
s <- s/mean(s)
shifted_log_counts <- log(counts/s + 1)
```

With these changes, rerun your script and see how your new plot compares to the old plot.

Your R script should now look something like this after addressing the “wrinkle”:

```
library(rsvd)
library(uwot)
set.seed(1)
barcodes <- read.csv("pancreas_barcodes.csv")
genes <- read.csv("pancreas_genes.csv")
counts <- readRDS("pancreas_counts.rds")
s <- rowSums(counts)
i <- which(s > 5000)
counts <- counts[i,]
barcodes <- barcodes[i,]
s <- rowSums(counts)
s <- s/mean(s)
shifted_log_counts <- log(counts/s + 1)
pca <- rpca(shifted_log_counts,k = 30)
res <- umap(pca$x,metric = "cosine",n_neighbors = 30,
            min_dist = 0.3,verbose = TRUE)
x <- res[,1]
y <- res[,2]
plot(x,y,pch = 20,xlab = "UMAP 1",ylab = "UMAP 2")
```

Exercise: Test your script by running it in a “clean” environment.

Recap of Part 3

A partial list of what I hope you will remember from Part 3:

1. Organize your project files in a folder, and develop an R script that records *what you have done*.
2. A data analysis always involves iteration: start with small, achievable aims, and don't be afraid to make mistakes.
3. While our ultimate goal is to make new discoveries, before we get there we first need to deeply understand the data.
4. Results that are incomplete or contain errors are always more useful than no results.

Programming challenge questions

These programming challenge questions build on the exploratory analysis of the pancreas data you have performed so far. Using UMAP, you discovered some structure underlying these data, and you improved your analysis by recognizing that the data need to be “normalized” before running the UMAP. The 2-d UMAP embedding clearly shows some clustering of the cells, with some clusters being more distinct than others. Here we will continue on the path of *data-driven discovery* and will try to understand the biological meaning of these clusters.

The first step is to *define* the clusters. Although there are ways to do this more systematically (and perhaps more rigorously), here we will take a simple approach: we will define the clusters visually based on UMAP projection of the cells onto 2-d space. While some clusters are very clearly distinct in this projection, others are not fully distinct, and so you will have to use your judgement to decide which “blob-like” structures are indeed “clusters”. (There is no single “correct” answer—I look forward to seeing what decisions you make.)

Some hints for this first step: I suggest creating a new object to keep track of the cluster labels for all the cells. One simple way to visualize the cluster labeling in the UMAP plot is to label the clusters by different colors. Then use the “col” argument in `plot()` to tell R to show the points in different colors. The `axis()` function with `tck = 1` could be helpful for adding a grid to the plot in order to figure out where exactly the clusters are in the UMAP projection. A final hint for this step is that you will probably want to repurpose some of what you did in the “facts about dog breeds” programming challenge (specifically, see the “The longest-living dog breeds” section) to identify the cells belonging to the different clusters. Another function that could be useful here is the “table” function. (Remember to use the R help feature to learn how to use these functions and what the different function arguments are for.)

In the second step, we will try to make sense of these clusters by performing a *differential expression analysis* that compares the expression of each gene *within* the cluster to the expression *outside* the cluster. There are different ways to quantify expression differences, but here we will take a simple approach using the shifted log counts. (Although this approach lacks statistical rigor, the calculations you will perform are actually not far off what is done in the statistical software tools designed to make these calculations rigorous.) The calculation will look something like this:

```
x <- colMeans(shifted_log_counts[cells_inside_cluster,])
y <- colMeans(shifted_log_counts[cells_outside_cluster,])
d <- x - y
```

These three lines of code compute the mean expression level (on the shifted log scale) for cells in the cluster and not in the cluster, and performs these calculations for each column of the shifted log counts matrix, then stores the results of these calculations in two vectors, `x` and `y`. (Remember, columns = genes.) Finally, the `d` is a vector containing the difference in the mean expression levels for each gene. (There is *a lot* of R's "invisible magic" in these lines of code.)

Remember that you will perform a differential expression analysis once per cluster. To help organize these analyses and easily make comparisons across the different clusters, I suggest storing all the results of your differential expression analyses in a single data structure, either a matrix or a data frame, in which the columns correspond to the clusters. Also, I recommend naming both the rows (genes) and the columns (clusters). For example, here is how to create a data frame in which the columns and rows are named:

```
dat <- data.frame(A = 1:3,B = 4:6,C = 7:9)
rownames(dat) <- c("dog","cat","ferret")
```

Once you have a data frame with named rows and columns, the rows and/or columns can be indexed by name, e.g.,

```
dat["cat","A"]
dat["cat",]
dat[,c("B","C")]
```

In the third step, we will make use of the differential expression analysis to interpret the clusters. We might expect that genes that show large increases in expression will be most relevant to understanding the biological role of the cells in the cluster, however this is complicated by the fact that typically there are many genes showing increases in expression, some which are quite relevant, but others less so, or perhaps their relevance is unknown. Therefore, to simplify this task, I suggest focussing on these genes with known relevance to pancreas cell types: *Gcg* (α cells); *Ins1*, *Ins2* (β cells); *Sst* (δ cells); *Ppy* (γ cells, also known as "pancreatic polypeptide" cells); *Krt19* (duct cells); *Pecam1* (endothelial cells); *Ccr5* (macrophages); and *Col1a1* (mesenchymal cells). Mice have two insulin genes (*Ins1*, *Ins2*) unlike humans which have only one (*INS*). Note that the mapping between pancreatic cell types and the clusters is not expected to be one-to-one; that is, some clusters may include multiple pancreatic cell types (probably cell types that are more similar to each other), and some pancreatic cell types may be represented by more than one cluster (in which this case the clusters might be capturing some interesting biological variation *within* the cell type, but understanding this within-cell-type variation is not within the scope of this programming challenge).

This is inevitably a simplified exploratory analysis, nonetheless my sincere hope is that Part 3 of Basic Computing has given you some idea of what is involved in making biological discoveries from data.